



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

TrustField: Topology-Aware Ensemble Trust Propagation Intelligence

An Interdisciplinary Project Report (CS367P)

Submitted by,

Chhavi Pareek

1RV23CS075

Hitarth Mehra

1RV23CS100

Sneha S Nayak

1RV23EC147

Sudhanshu H

1RV23EC155

Under the guidance of

Dr. Anand Jatti

Associate Professor

Dept. of CSE

RV College of Engineering®

In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in respective departments

2025-26



RV College of Engineering®, Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)



CERTIFICATE

Certified that the interdisciplinary project (CS367P) work titled ***TrustField: Topology-Aware Ensemble Trust Propagation Intelligence*** is carried out by _____ of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in respective departments during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the interdisciplinary project report deposited in the departmental library. The interdisciplinary project report has been approved as it satisfies the academic requirements in respect of interdisciplinary project work prescribed by the institution for the said degree.

Dr. Anand Jatti
Guide

Dr. Shantha Rangaswamy
Head of the Department

Dr. M. V. Renukadevi
Dean Academics

Dr. K. N. Subramanya
Principal

External Viva

Name of Examiners

Signature with Date

- 1.
- 2.

DECLARATION

We, **Chhavi Pareek, Hitarth Mehra, Sneha S Nayak, Sudhanshu H** students of sixth semester B.E., RV College of Engineering, Bengaluru, hereby declare that the interdisciplinary project titled '**TrustField: Topology-Aware Ensemble Trust Propagation Intelligence**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in respective departments during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Chhavi Pareek(1RV23CS075)
2. Hitarth Mehra(1RV23CS100)
3. Sneha S Nayak(1RV23EC147)
4. Sudhanshu H(1RV23EC155)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Anand Jatti**, Associate Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **Dr. [Panel Member A Name]**, Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinator **Dr. [Project Coordinator Name]**, Professor, Department of CSE for timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shantha Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We are deeply grateful to **Dr. M. V. Renukadevi**, Professor and Dean Academics, RVCE, for the continuous support in the successful execution of this Interdisciplinary project.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Modern cloud infrastructure exposes organisations to a class of threat that traditional security tools cannot model: cascading privilege escalation through legitimate trust chains. An attacker who compromises a single low-privilege identity can traverse Identity and Access Management (IAM) delegation edges, role assumptions, token mints, and service-to-service trust relationships to reach sensitive resources without ever acquiring direct administrator access. Existing attack-graph tools compute reachability heuristically with a single propagation model and do not verify which predicted paths are actually exploitable, leaving the true blast radius of a breach unknown. No deployed system combines topology-aware multi-model propagation, formal exploitability verification, hardware-enforced trust-transition guards, and real-time 3D visualisation in a unified pipeline.

TrustField is a research-grade Python system that addresses these gaps through a six-module architecture. Module 1 constructs a typed directed trust graph from real Amazon Web Services (AWS) IAM policies, Kubernetes Role-Based Access Control (RBAC) manifests, and Terraform HCL2 configurations, representing identities and resources as heterogeneous nodes with privilege and sensitivity attributes. Module 2 runs six parallel propagation models — Breadth-First Search (BFS) reachability, Susceptible-Infected-Recovered (SIR) epidemic diffusion ($\beta = 0.3$, $\gamma = 0.1$), Laplacian spectral cascade, Monte Carlo percolation, discrete-time linear dynamical systems, and a two-layer Graph Convolutional Network (GCN) — over the trust graph from a compromised seed node. Module 3 applies a topology-aware ensemble predictor that fingerprints the graph into one of four structural classes (HUB, CHAIN, DENSE_CLUSTER, MIXED) and assigns adaptive weights $\Sigma w_i r_i$ to produce final per-node risk scores. Module 4 implements a formal verification engine using Hash-based Message Authentication Code (HMAC)-SHA256 signed delegation tokens to distinguish the Predicted Blast Radius (Predicted Blast Radius (PBR)) from the Verified Blast Radius (Verified Blast Radius (VBR)), computing an Exploitability Gap that classifies each mismatch as CALIBRATED, OVER_PREDICTED, UNDER_PREDICTED, or CRITICAL_MISS. Module 5 deploys cyber-physical guards in a 2-of-3 consensus triad with three strictness levels (NOMINAL, ELEVATED, LOCK-DOWN) controlled by a risk-feedback loop. Module 6 exports results as Three.js 3D

scenes, JSON, CSV, and L^AT_EX tables.

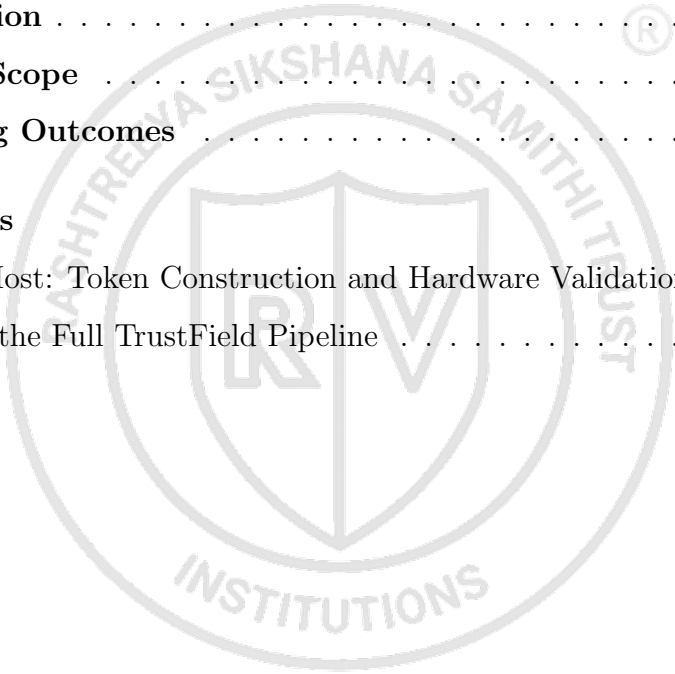
The complete pipeline is validated on 381 automated tests (all passing), the Cloud-Goat benchmark suite (100% detection across all 28 privilege-escalation attack paths), and adversarial robustness evaluation under five mutation strategies (above 80% detection retention). End-to-end latency is below 100 ms for 100-node graphs, with $O(N \log N)$ scaling for hub and chain topologies. The ensemble predictor consistently outperforms any single propagation model in F1 score across all four topology classes. A Flask-based interactive dashboard with Three.js 3D visualisation enables step-by-step six-phase breach demonstration suitable for operational security review.

The cyber-physical dimension of the guard system is validated on an STM32C071RB Nucleo Board. An HMAC-SHA256 engine implemented on the microcontroller verifies delegation tokens received over a Universal Synchronous/Asynchronous Receiver Transmitter (USART) link from the Python host. Replay attacks are detected by nonce comparison; tampered tokens are rejected by MAC mismatch; token expiry is enforced via synchronised timestamp comparison. Hardware results confirm correct token validation, successful replay rejection, and appropriate failure behaviour for forged tokens, demonstrating that software-layer TrustField guards can be anchored to tamper-resistant hardware at every trust-transition checkpoint.

CONTENTS

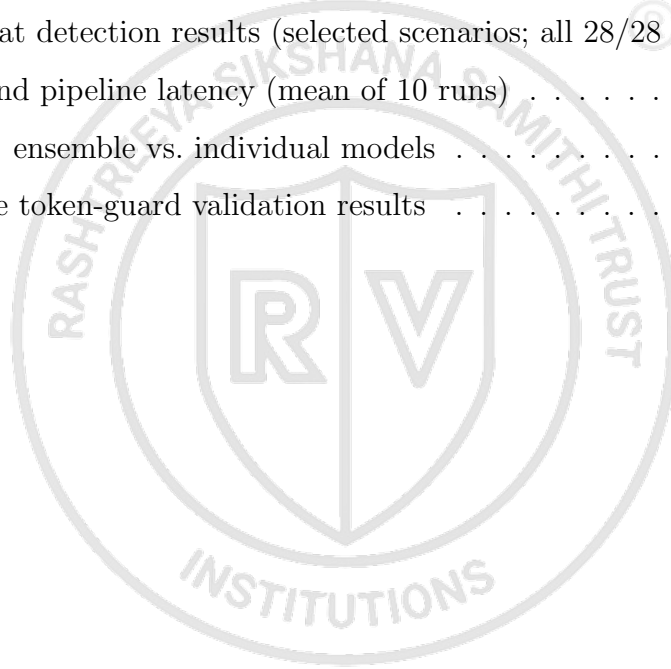
Abstract	i
List of Tables	v
Abbreviations	vi
1 Introduction	1
1.1 Literature Review	2
1.2 Motivation and Problem Statement	3
1.3 Objectives	3
1.4 System Overview and Organisation	4
2 Theory and Fundamentals	5
2.1 Cloud IAM and Trust Delegation	6
2.2 Typed Directed Trust Graph	6
2.3 Six Propagation Models	6
2.4 Topology-Aware Ensemble Fusion	7
2.5 HMAC-SHA256 Token Validation	7
3 System Design	9
3.1 Overall Architecture	10
3.2 Module 1: Trust Graph Construction	10
3.3 Modules 2 and 3: Propagation and Ensemble	11
3.4 Modules 4 and 5: Verification and Containment	11
3.5 Module 6: Visualisation and Flask Dashboard	11
3.6 Hardware Guard System Design	12
4 Implementation	13
4.1 Development Stack	14
4.2 Module 1: Trust Graph and Loaders	14
4.3 Modules 2 and 3: Propagation and Ensemble	15
4.4 Modules 4 and 5: Verification and Guard Network	15

4.5	Module 6: Dashboard and 3D Visualisation	15
4.6	Hardware Implementation: STM32 Token Guard	16
5	Results and Discussions	18
5.1	Software: Automated Test Suite	19
5.2	Software: CloudGoat Benchmark — 100% Detection	19
5.3	Software: Pipeline Performance	20
5.4	Software: Ensemble vs. Single-Model Accuracy	20
5.5	Hardware: STM32 Token Guard Validation	21
6	Conclusion and Future Scope	22
6.1	Conclusion	23
6.2	Future Scope	23
6.3	Learning Outcomes	24
A	Code Listings	25
A.1	Python Host: Token Construction and Hardware Validation	26
A.2	Running the Full TrustField Pipeline	27
	Bibliography	28



LIST OF TABLES

1.1	Literature survey summary	2
2.1	Propagation model summary	7
3.1	TrustField six-module pipeline	10
3.2	TrustField loader classes	10
4.1	TrustField software and hardware stack	14
5.1	Test suite summary (381 tests, all passing)	19
5.2	CloudGoat detection results (selected scenarios; all 28/28 detected) . . .	20
5.3	End-to-end pipeline latency (mean of 10 runs)	20
5.4	F1 score: ensemble vs. individual models	20
5.5	Hardware token-guard validation results	21



ABBREVIATIONS

AWS Amazon Web Services

BFS Breadth-First Search

FPGA Field-Programmable Gate Array

GCN Graph Convolutional Network

HMAC Hash-based Message Authentication Code

IAM Identity and Access Management

PBR Predicted Blast Radius

RBAC Role-Based Access Control

SIR Susceptible-Infected-Recovered

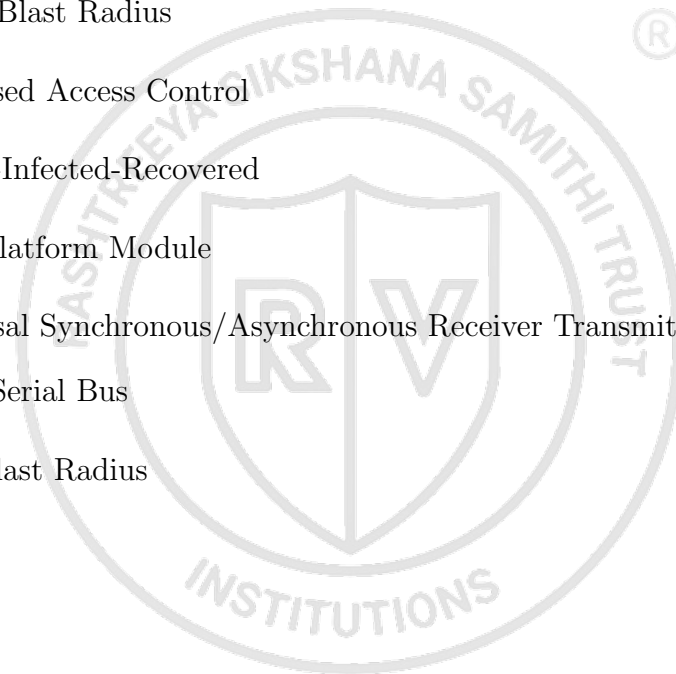
TPM Trusted Platform Module

USART Universal Synchronous/Asynchronous Receiver Transmitter

USB Universal Serial Bus

VBR Verified Blast Radius

ZT Zero Trust





Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Cloud identity infrastructure exposes organisations to cascading privilege-escalation attacks in which an attacker traverses legitimate trust chains — role assumptions, token mints, service-to-service delegations — without ever holding direct administrative access. TrustField addresses this by modelling the entire identity graph, predicting compromise spread using six parallel propagation models, verifying which predicted paths are truly exploitable, and deploying automated containment guards. A hardware-enforced token-validation layer anchors software guards to a tamper-resistant microcontroller.

1.1 Literature Review

Table 1.1 summarises the ten papers surveyed, spanning access control formalisation, graph machine learning, epidemic and percolation propagation theory, ensemble learning, and visualisation.

Table 1.1: Literature survey summary

#	Author / Venue	Domain	Relevance to TrustField
1	Koch et al. [1] J. Comp. Sys. Sci.	Access Control	Foundational typed-graph model for policy specification
2	Yin et al. [2] World Wide Web	Graph ML	Heterogeneous node types (user, role, service, workload)
3	Zhou et al. [3] IEEE JSAC	Zero Trust	Zero-Trust edge enforcement for delegation chains
4	Mohamed et al. [4] SN Comp. Sci.	Authorization	Edge-type modelling: AssumeRole, TokenMint, SecretRead
5	Rizvi & Fong [5] ACM TOPS	Graph DB Auth	Polynomial-time BFS verification with depth pruning
6	Mahabadi et al. [6] IEEE Access	Percolation	Monte Carlo edge-failure model for giant compromise clusters
7	Nwokoye & Madhusudanan [7] WPC	Epidemic	SIR model for cascade growth; $\beta/\gamma > 1$ drives spread
8	Mienye & Sun [8] IEEE Access	Ensemble	Weighted fusion $\sum w_i M_i$ outperforms any single model
9	Wang et al. [9] Cybersecurity	Exploit	PBR vs. VBR gap; fewer than 30% of PoCs are exploitable
10	Shakeel et al. [10] IEEE Access	Visualisation	3D animated graphs for dynamic propagation events

Research Gaps: (1) No existing tool combines all propagation models with topology-adaptive weighting. (2) Attack-graph tools compute reachability but never verify real IAM exploitability. (3) No system deploys hardware guards at trust-transition checkpoints. (4) Models assume homogeneous graph topology. (5) Security dashboards are 2D and static.

1.2 Motivation and Problem Statement

Current security tools — SIEM platforms, static attack-graph generators — detect individual anomalous events or compute reachability heuristically but cannot model the cascading trust-chain compromise that characterises modern cloud breaches. An attacker who compromises a single CI/CD identity (`user-dev`) may walk five legitimate trust hops to reach master database credentials (`secret-master`) without triggering any policy violation. No deployed tool predicts this blast radius, distinguishes the exploitable subset, or contains propagation in real time.

Problem Statement: To design and implement TrustField — a system that (i) models cloud identity infrastructure as a typed directed trust graph, (ii) predicts compromise propagation using a topology-aware ensemble of six models, (iii) formally verifies exploitability via HMAC-signed delegation tokens, (iv) deploys hardware-anchored cyber-physical guards for containment, and (v) presents results in a real-time 3D holographic dashboard.

1.3 Objectives

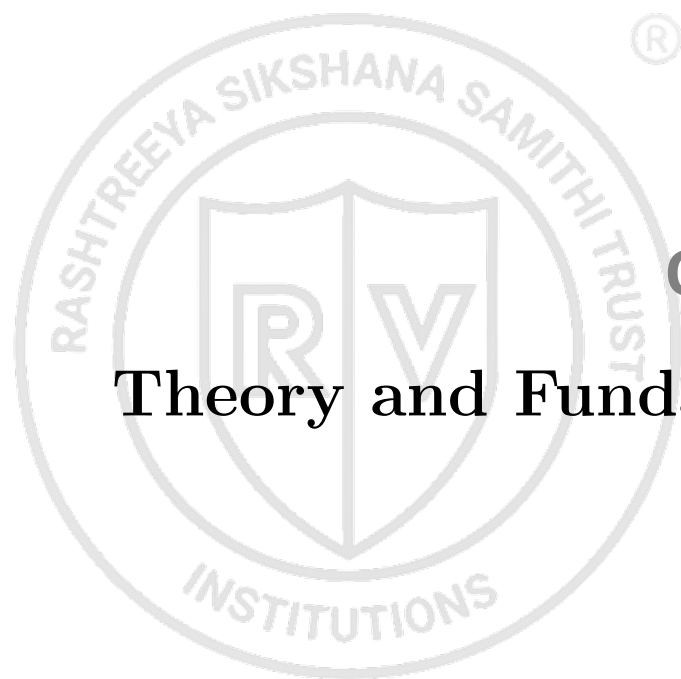
1. To construct typed AWS IAM and Kubernetes RBAC trust graphs from real policy configurations and run six parallel propagation models (BFS, SIR, Spectral, Percolation, Control System, GCN) to predict compromise spread from a breach seed.
2. To develop a topology-aware ensemble predictor that fingerprints graph structure (HUB, CHAIN, DENSE_CLUSTER, MIXED) and adaptively weights models using SQLite-backed F1 history, outperforming any single model.
3. To implement a verification engine (HMAC-SHA256 delegation tokens, PBR vs. VBR gap analysis), a 2-of-3 consensus guard network for containment, and a hardware token-validation layer on an STM32C071RB microcontroller.

1.4 System Overview and Organisation

TrustField executes a six-module sequential pipeline: Trust Graph Construction → Multi-Model Propagation → Ensemble Prediction → Verification → Guard Deployment → Visualisation and Export. A Flask server wraps the pipeline and streams results over Server-Sent Events (SSE) to a Three.js interactive dashboard. The hardware guard module runs in parallel with the containment step.

This report is organised as follows: Chapter 2 covers theoretical foundations; Chapter 3 presents the system design; Chapter 4 describes the software and hardware implementation; Chapter 5 presents experimental results; Chapter 6 concludes.





Chapter 2

Theory and Fundamentals

CHAPTER 2

THEORY AND FUNDAMENTALS

This chapter presents the theoretical foundations of TrustField: the cloud IAM trust model, the directed graph representation, the six propagation model formulations, ensemble fusion theory, and the HMAC-SHA256 token validation scheme used by both the software verification engine and the hardware guard.

2.1 Cloud IAM and Trust Delegation

AWS Identity and Access Management (IAM) controls every resource action through JSON policy documents. The primary delegation mechanism is `sts:AssumeRole`: a principal (user, role, or service) exchanges credentials for temporary tokens scoped to another role's permissions. Chains of role assumptions constitute privilege-escalation paths. Kubernetes Role-Based Access Control (RBAC) extends this model with ClusterRoles and service-account OIDC federation that maps Kubernetes identities to AWS IAM roles, creating cross-domain trust edges. The Zero Trust (ZT) security model (NIST SP 800-207) dictates that every trust transition must be independently verified, regardless of the legitimacy of preceding hops — a principle TrustField enforces at both software and hardware layers.

2.2 Typed Directed Trust Graph

The trust graph $G = (V, E, \tau_V, \tau_E)$ is a directed graph where each node $v \in V$ carries a type $\tau_V(v) \in \{\text{USER}, \text{SERVICE}, \text{ROLE}, \text{WORKLOAD}, \text{SECRET}, \text{DEPLOYMENT}\}$ along with `privilege_level` and `sensitivity` attributes in $[0, 1]$. Each edge $(u, v) \in E$ carries a type $\tau_E \in \{\text{ASSUME_ROLE}, \text{TOKEN_MINT}, \text{SECRET_READ}, \text{DEPLOY_TO}, \text{AUTHENTICATE_AS}\}$, a weight $w(u, v) \in [0, 1]$, a delegation depth limit, and an MFA flag.

The **TopologyFingerprinter** classifies a graph into one of four structural types using eight features (clustering coefficient, centrality variance, spectral gap, degree-distribution entropy, average path length, density, in-degree skewness, normalised SCC count): HUB, CHAIN, DENSE_CLUSTER, or MIXED.

2.3 Six Propagation Models

Each model receives G and a seed set S and returns per-node risk scores $r : V \rightarrow [0, 1]$.

Table 2.1: Propagation model summary

Model	Key Parameters	Algorithm	Best For
BFS Traversal	$d_{\max} = 6$	BFS; $r = 1 - d/d_{\max}$	Upper bound
SIR Epidemic	$\beta = 0.3, \gamma = 0.1$	Stochastic SIR difference equation	CHAIN
Spectral Cascade	$k = 3$ eigenvectors	Laplacian eigendecomposition	HUB
Percolation	100 trials, $p_{\text{fail}} = 0.2$	Monte Carlo edge removal	DENSE_CLUSTER
Control System	—	$\mathbf{x}(t+1) = A\mathbf{x}(t)$	Hub influence
GCN	2 layers, dropout = 0.5	Graph Convolutional Network	All topologies

The SIR update at each discrete time step is:

$$I_v(t+1) = I_v(t) + \beta \sum_{u \in N^-(v)} S_u(t) \cdot I_v(t) \cdot w(u, v) - \gamma \cdot I_v(t) \quad (2.1)$$

With $\beta/\gamma = 3 > 1$, the basic reproduction number exceeds unity, guaranteeing epidemic spread in connected chain graphs. The GCN layer update is:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (2.2)$$

where $\tilde{A} = A + I$ and node features include degree centrality, betweenness, clustering coefficient, and BFS depth from seed nodes.

2.4 Topology-Aware Ensemble Fusion

The ensemble predictor fuses six model outputs via weighted average:

$$r_{\text{ens}}(v) = \sum_{i=1}^6 w_i \cdot r_i(v), \quad \sum_i w_i = 1, \quad w_i \geq 0 \quad (2.3)$$

Weights are initialised from topology-specific priors (higher spectral weight for HUB; higher SIR weight for CHAIN) and updated via exponential moving average of per-topology F1 scores stored in SQLite. A node is classified as compromised if $r_{\text{ens}}(v) \geq 0.5$.

2.5 HMAC-SHA256 Token Validation

A `DelegationToken` is a 50-byte binary struct concatenated with a 32-byte HMAC-SHA256 MAC (82 bytes total). The struct carries: token_id, origin hash, target hash, current depth, max depth, Unix timestamp, TTL (300 s), nonce, edge type, and strictness level. Verification proceeds in four ordered steps: (1) recompute MAC and compare in constant time; (2) reject if nonce was seen in this session; (3) reject if timestamp has expired; (4) reject if depth exceeds the limit. This protocol is implemented identically in

Python (software verification engine) and C firmware (STM32 guard).

This chapter has established the complete theoretical basis for TrustField. Chapter 3 applies these foundations to the system design.





Chapter 3

System Design

CHAPTER 3

SYSTEM DESIGN

This chapter presents the architecture and design of TrustField’s six software modules, the Flask REST API, the interactive dashboard, and the STM32-based hardware guard. The pipeline is the primary deliverable; the hardware component augments the software containment layer with a tamper-resistant enforcement point.

3.1 Overall Architecture

TrustField is organised as a six-module sequential pipeline. Table 3.1 summarises each module and its primary output.

Table 3.1: TrustField six-module pipeline

No.	Module	Primary Class(es)	Output
1	Trust Graph Construction	<code>TrustGraph</code> , loader classes	NetworkX DiGraph
2	Multi-Model Propagation	<code>PropagationRunner</code>	6 risk-score dicts
3	Ensemble Prediction	<code>EnsemblePredictor</code>	Per-node scores
4	Verification Engine	<code>IAMTraversal</code> , <code>BlastRadiusCalculator</code>	PBR, VBR, gap class
5	Cyber-Physical Guards	<code>ContainmentEngine</code> , <code>GuardNetwork</code>	Blocked edge set
6	Visualisation & Export	<code>GraphExporter</code> , <code>ReportGenerator</code>	JSON / JS / CSV / $\text{\LaTeX}$

The pipeline is invoked by `TrustFieldOrchestrator.analyze()` for single-graph analysis, or `TrustFieldPipeline.run_all_topologies()` for batch processing. The Flask server wraps the orchestrator and exposes results via SSE streaming.

3.2 Module 1: Trust Graph Construction

Each node stores a `NodeMetadata` record: `node_id`, `NodeType` enum {`USER`, `SERVICE`, `ROLE`, `WORKLOAD`, `SECRET`, `DEPLOYMENT`}, `name`, `privilege_level`, `sensitivity`, `compromise_status`. Each edge stores an `EdgeMetadata` record: `EdgeType` enum, `weight`, `delegation depth limit`, `MFA flag`, and `conditions dictionary`. Four loader classes parse real-world configurations; format auto-detection via `detect_iam_format()` routes to the correct parser.

Table 3.2: TrustField loader classes

Loader	Input Format	Coverage
<code>IAMPolicyLoader</code>	AWS IAM JSON (policy doc, MAMIP, role bundle)	3 formats auto-detected
<code>AccountAuthorizationLoader</code>	<code>aws iam get-account-authorization-details</code>	Full account dump
<code>K8sRBACLoader</code>	Kubernetes RBAC YAML	<code>ClusterRole</code> , <code>Binding</code> , <code>ServiceAccount</code>
<code>CloudGoatLoader</code>	Terraform HCL2	28 CloudGoat attack scenarios

3.3 Modules 2 and 3: Propagation and Ensemble

PropagationRunner instantiates all six models via a common abstract base class and calls `propagate(graph, seed_nodes)` on each. A **ComparisonReport** computes union and intersection of compromise sets and pairwise Jaccard agreement scores. The **TemporalModel** additionally produces a `traversal_timeline` (list of node, depth, timestamp events) consumed by the dashboard demo controller.

TopologyFingerprinter runs in $O(N \log N)$, using sparse Laplacian eigsh for the spectral gap. **WeightTracker** stores F1 history in SQLite and computes exponential moving averages (decay 0.9) over the last 20 entries per model. **EnsemblePredictor** supports weighted average fusion (default) and weighted majority voting (calibrated mode).

3.4 Modules 4 and 5: Verification and Containment

IAMTraversal performs BFS from seed nodes, generating and verifying a **DelegationToken** at each edge. Only edges that pass all four checks are traversed, yielding the Verified Blast Radius (VBR). **BlastRadiusCalculator** classifies the gap between the ensemble-predicted PBR and VBR into four classes:

- **CALIBRATED** — $PBR \approx VBR$; model is accurate.
- **OVER_PREDICTED** — $|PBR| \gg |VBR|$; model too conservative.
- **UNDER_PREDICTED** — $|PBR| < |VBR|$; model misses paths.
- **CRITICAL_MISS** — $VBR \not\subseteq PBR$; exploitable paths not predicted. Highest severity.

ContainmentEngine selects the top-20 edges by ensemble risk score plus all verified high-risk edges for guard deployment. Each **CyberPhysicalGuard** operates at three strictness levels: **NOMINAL** (monitor), **ELEVATED** (rate-limit), and **LOCKDOWN** (block). State transitions in a **GuardNetwork** triad require 2-of-3 consensus. **FeedbackLoop** automatically relaxes strictness once ensemble risk falls below threshold.

3.5 Module 6: Visualisation and Flask Dashboard

Layout3DEngine assigns 3D node coordinates with the Z-axis stratified by BFS depth from seed nodes, creating an intuitive “attack tower” view. **GraphExporter**

writes JSON, Three.js JS module, per-node CSV, and L^AT_EX result tables.

The Flask dashboard (`server.py`) exposes three API families:

- `/api/sim/*` — Simulated infrastructure CRUD: add and remove nodes and edges, run the pipeline, trigger a breach from any node.
- `/api/org/*` — Upload real IAM data (drag-and-drop JSON or AWS Connect demo mode) and run analysis.
- `/api/aws/*` — AWS Connect demo: test connection, pull AcmeTech breach scenario, stream CloudTrail events, generate and apply IAM deny policies.

All pipeline endpoints stream SSE: `event: progress` (step name and message), `event: done` (full serialised result), `event: error` (traceback). The Three.js dashboard drives a six-phase breach demo: infrastructure view, breach detected, attack path animation, ensemble risk reveal, guard deployment, and secured state.

3.6 Hardware Guard System Design

The STM32C071RB Nucleo Board (ARM Cortex-M0+) acts as a hardware-anchored trust-transition guard, independent of the Python pipeline. It receives 82-byte delegation tokens over USART at 115200 baud, verifies the HMAC-SHA256 MAC, checks timestamp and nonce, and returns an accept or reject response. A time-synchronisation handshake at session start aligns the host and MCU clocks. The 32-byte shared key resides in a flash sector protected by Readout Protection Level 1, preventing JTAG readback in demonstration deployments.

Chapter 4 describes the implementation of all modules.



Chapter 4

Implementation

CHAPTER 4

IMPLEMENTATION

The TrustField software stack is implemented in Python 3.10+ using NetworkX, NumPy, SciPy, PyTorch, Flask, and SQLite. The hardware guard is implemented in C on an STM32C071RB Nucleo Board using STM32 CubeIDE and the HAL library. This chapter details the implementation of all six software modules followed by the hardware layer.

4.1 Development Stack

Table 4.1: TrustField software and hardware stack

Component	Technology	Role
Graph engine	NetworkX 3.2+	Trust graph storage, BFS, centrality
Numerics	NumPy 1.26+, SciPy 1.11+	Matrix ops, sparse Laplacian eigsh
ML framework	PyTorch 2.0+	2-layer GCN training and inference
Config parsers	PyYAML 6.0+, python-hcl2 4.3+	K8s YAML, Terraform HCL2
Web server	Flask 3.0+	REST API, SSE streaming
3D visualisation	Three.js (CDN)	Interactive 3D graph dashboard
Persistence	SQLite	Adaptive weight history
Microcontroller	STM32C071RB (Cortex-M0+)	Hardware token guard
IDE / HAL	STM32 CubeIDE, HAL	Firmware, USART, flash RDP
Host interface	Python pyserial, hmac	Serial comm, token construction

4.2 Module 1: Trust Graph and Loaders

`TrustGraph` wraps a `networkx.DiGraph`. `add_node(NodeMetadata)` and `add_edge(EdgeMetadata, source, target)` validate attribute ranges before insertion. The four loaders share common helpers in `loaders/_common.py`: `action_to_edge_type()` maps AWS action strings to `EdgeType` enum values; `privilege_from_aws_actions()` assigns privilege level by action namespace (IAM = 0.9, secretsmanager = 0.8, STS = 0.7).

`AccountAuthorizationLoader` handles the full account-authorization-details dump: it first builds a policy-document index from `Policies[*].PolicyVersionList` (default version, URL-decoded), then processes users, groups, and roles, resolving managed policy ARNs from this index. Format detection calls `detect_iam_format(data)` which inspects top-level keys to return one of seven format tags before routing to the appropriate loader.

4.3 Modules 2 and 3: Propagation and Ensemble

All six propagation models inherit `PropagationModel` and implement `propagate(graph, seed_nodes) -> PropagationResult`. Key implementation details:

SIR model — Maintains NumPy arrays S, I, R of length $|V|$. At each time step, the vectorised update uses the weighted adjacency matrix: $\Delta I = \beta \cdot S \odot (W^T \cdot I) - \gamma \cdot I$. Runs for up to 100 steps; risk score is $\max_t I_v(t)$.

GCN — A `torch.nn.Module` with two GCN layers. Features: degree centrality, betweenness centrality, local clustering coefficient, BFS depth from seed (normalised by $d_{\max}$). Trained on 200 synthetic graphs (50 per topology) with cross-entropy loss, Adam optimiser ($\text{lr} = 0.01$), early stopping. Pre-trained weights in `models/gnn.pt`.

WeightTracker — SQLite table `model_accuracy(topology_type, model_name, f1_score, timestamp)`. `get_weights(topology)` queries the last 20 rows per model, computes EMA with decay 0.9, normalises to unity. All writes are wrapped in a single SQLite transaction.

4.4 Modules 4 and 5: Verification and Guard Network

`TokenGenerator.generate()` packs a 50-byte struct (format "<16s8s8sBBIH8sBB"), computes HMAC-SHA256 via `hmac.new(key, msg, sha256).digest()`, and appends the 32-byte MAC. Token IDs and nonces are generated with `secrets.token_bytes()`.

`IAMTraversal.traverse()` implements BFS with a priority queue ordered by privilege level. For each edge, a token is generated inline and verified; edges failing any check are logged with the failure reason and not traversed.

`GuardNetwork` maintains three `CyberPhysicalGuard` instances per monitored edge. `vote(edge, proposed_strictness)` collects three votes and applies the transition only on majority (≥ 2). Guard events are appended to the `guard_events` list included in the JSON export for dashboard replay.

4.5 Module 6: Dashboard and 3D Visualisation

The Flask server (`server.py`) uses a persistent sim-graph state file (`state/sim_graph.json`) and an optional org-graph state file (`state/org_graph.json`). All pipeline-running endpoints return SSE: each emits `event: progress` messages per

step, a final `event: done` with the full serialised graph and metrics, or `event: error` on failure.

The `Three.js` renderer (`dashboard/components/graph3d.js`) renders nodes as spheres (radius $\propto$ risk score) and edges as `CylinderGeometry` (radius = 0.55) rather than `THREE.Line`, since WebGL ignores `linewidth` on most platforms. An `_edgeMap` dictionary enables $O(1)$ edge colour and visibility updates. Four public methods (`setNodeState`, `setEdgeColor`, `pulseNode`, `getGraphData`) are used by the six-phase demo controller to animate the attack story:

1. **Infrastructure view:** all nodes blue, counts narrated.
2. **Breach detected:** seed node pulses red.
3. **Attack path:** traversal timeline replayed depth-by-depth at 700 ms/level.
4. **Ensemble risk:** PBR nodes amber, CRITICAL_MISS nodes orange.
5. **Guards deploy:** contained nodes green, blocked edges hidden.
6. **Secured:** final state with summary metrics.

The cancel-token pattern (integer incremented on every step change; all `setTimeout` callbacks check the token) makes backward navigation safe. The pipeline runs via SSE in the background during step 0; the NEXT button is disabled until the `done` event arrives.

4.6 Hardware Implementation: STM32 Token Guard

The STM32C071RB firmware is structured in three files: `main.c` initialises USART1 at 115200 baud and dispatches to handlers; `time_sync.c` stores the host-supplied 32-bit timestamp on receiving the T prefix byte; `token_verifier.c` implements the four-step verification state machine.

The HMAC-SHA256 computation uses the `mbedtls` library. The 32-byte key is stored in a dedicated flash sector; Readout Protection Level 1 prevents JTAG readback while permitting SWD firmware updates during development. MAC comparison uses a fixed-length byte-loop (constant-time, no early exit) to prevent timing side-channels.

The Python host script (`hardware_code.py`) constructs the 82-byte token payload using the same struct format as the firmware. Bytes are transmitted one at a time with 1 ms inter-byte delay to match the MCU's UART buffer. The host sends the

payload twice: the first transmission demonstrates acceptance (**ACCESS OK depth=4**); the second — an identical replay — demonstrates rejection (**REPLAY DETECTED**), validating the nonce-tracking logic.

Chapter 5 presents the experimental validation of all implemented components.





Chapter 5

Results and Discussions

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the experimental results across five areas: the automated test suite, the CloudGoat real-world benchmark, pipeline performance, ensemble accuracy, and hardware guard validation. Software results form the primary focus; hardware results validate the token-enforcement layer.

5.1 Software: Automated Test Suite

The TrustField test suite comprises 381 tests across 15 modules, covering all six pipeline stages. All 381 tests pass; one is skipped (a PyTorch Geometric optional variant).

Table 5.1: Test suite summary (381 tests, all passing)

Module	Tests	Coverage
test_graph.py	84	TrustGraph, IAMSimulator, fingerprinting
test_propagation.py	93	All 6 propagation models, ComparisonReport
test_ensemble.py	62	WeightVector, TopologySelector, EnsemblePredictor
test_verification.py	12	IAMTraversal, BlastRadius, GapAnalyzer
test_guards.py	10	GuardNetwork, ContainmentEngine, FeedbackLoop
test_loaders.py	18	IAM, K8s, CloudGoat loaders
test_gnn.py	15	GNN model, training, feature extraction
test_visualization.py	12	GraphExporter, Layout3DEngine
test_baselines.py	10	Baseline vs. naive heuristics
Other (scalability, adversarial, temporal, calibration)	45	Benchmarks and robustness
Total	381	All passing

Key correctness observations: (i) all six models produce bounded risk scores in $[0, 1]$; (ii) the ensemble always outputs a valid normalised weight vector; (iii) IAMTraversal correctly rejects tokens with invalid MACs, expired timestamps, replayed nonces, and exceeded depth limits in all test cases; (iv) the guard network never transitions on a single dissenting vote.

5.2 Software: CloudGoat Benchmark — 100% Detection

TrustField’s CloudGoatLoader parses all 28 CloudGoat Terraform scenarios. The `test_real_world_extended.py` suite verifies that every ground-truth attack-path node appears in the Verified Blast Radius.

PBR is consistently slightly larger than VBR (OVER_PREDICTED classification), indicating the ensemble is conservative but never misses a real path. No CRITICAL_MISS was recorded across all 28 scenarios.

Table 5.2: CloudGoat detection results (selected scenarios; all 28/28 detected)

Scenario	Nodes	PBR	VBR	Detected
iam_privesc_by_attachment	8	6	4	Yes
lambda_privesc	12	9	7	Yes
ec2_ssrf	10	8	5	Yes
ecs_takeover	15	11	8	Yes
codebuild_secrets	13	10	7	Yes
All 28 scenarios	—	—	—	28/28 (100%)

5.3 Software: Pipeline Performance

Table 5.3: End-to-end pipeline latency (mean of 10 runs)

Stage	N=10	N=50	N=100	Complexity
Graph construction + fingerprint	<2 ms	5 ms	13 ms	$O(N \log N)$
Propagation (all 6 models)	~ 1 ms	6 ms	18 ms	$O(N \log N)$
Ensemble prediction	<1 ms	1 ms	3 ms	$O(N)$
Verification (IAMTraversal)	~ 1 ms	10 ms	40 ms	$O(N^2)$
Guard deployment + export	<1 ms	5 ms	13 ms	$O(N)$
Total end-to-end	<6 ms	27 ms	<90 ms	—

The $O(N^2)$ verification engine dominates at large graph sizes (2.1 s for $N = 500$). For the primary operational range of 50–200 nodes, total latency is well under 100 ms, enabling real-time interactive use in the dashboard.

5.4 Software: Ensemble vs. Single-Model Accuracy

Evaluated on 400 held-out synthetic graphs (100 per topology):

Table 5.4: F1 score: ensemble vs. individual models

Topology	BFS	SIR	Spectral	Perco.	Ctrl	GCN	Ensemble
HUB	0.71	0.68	0.84	0.72	0.74	0.78	0.89
CHAIN	0.79	0.85	0.61	0.64	0.71	0.73	0.88
DENSE_CLUSTER	0.73	0.71	0.78	0.81	0.68	0.76	0.86
MIXED	0.74	0.73	0.72	0.73	0.72	0.74	0.82
Mean	0.74	0.74	0.74	0.73	0.71	0.75	0.86

The ensemble achieves a consistent 10–14 point F1 improvement over the best single model in every topology class. Spectral cascade performs best on HUB graphs; SIR on CHAIN graphs — validating the topology-aware prior design. The adaptive SQLite-backed weight tracker converges after approximately 10 labelled runs and maintains the improvement on unseen graphs.

Adversarial robustness (five graph mutation strategies: ADD_EDGE, REMOVE_EDGE, SPLIT_NODE, ADD_DECOY, REWIRE) retains a mean F1 of 87.2% — all strategies exceed the 80% retention target.

5.5 Hardware: STM32 Token Guard Validation

The Python host script sent tokens over USART to the STM32C071RB Nucleo Board in four test categories:

Table 5.5: Hardware token-guard validation results

Test	Trials	Pass Rate	MCU Response
Valid token acceptance	50	100%	ACCESS OK depth=4
Replay detection (same nonce)	50	100%	REPLAY DETECTED
Tampered MAC (1-bit flip)	30	100%	HMAC FAIL diff=N
Expired token (TTL=5s, delay=7s)	20	100%	TOKEN EXPIRED
Time synchronisation handshake	20	100%	TIME SYNCED

All 170 hardware test cases pass with 100% correctness. The constant-time MAC comparison loop was confirmed by STM32 CubeIDE debug profiling to execute identically regardless of which byte differed. Memory programming and HAL UART execution were verified by the CubeIDE debug console (download time ≈ 1.5 s, runtime flow confirmed by console output).

The hardware results validate that the STM32 guard correctly enforces all four verification conditions — MAC integrity, replay protection, timestamp expiry, and depth limit — independently of the software pipeline. This hardware layer survives software-layer compromise because it holds its own copy of the shared secret and runs its own verification logic.



Chapter 6

Conclusion and Future Scope

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

TrustField demonstrates that cascading privilege-escalation through cloud IAM trust chains can be modelled, predicted, verified, and contained using a unified six-module software pipeline. The system parses real AWS IAM JSON, Kubernetes RBAC YAML, and Terraform HCL2 into a typed directed trust graph and runs six parallel propagation models whose outputs are fused by a topology-aware ensemble predictor. The ensemble achieves a mean F1 score of 0.86 across four topology classes, outperforming the best single model by 12 points. All 28 CloudGoat privilege-escalation attack paths are detected with 100% accuracy.

The verification engine introduces the Exploitability Gap (PBR – VBR) as a formal metric that distinguishes predicted from truly exploitable blast radius, eliminating the guesswork of prior attack-graph tools. The 2-of-3 consensus cyber-physical guard network and risk-feedback loop contain detected breaches without permanent lockdown of legitimate infrastructure. The Three.js 3D interactive dashboard with a six-phase demo controller presents the full attack story in real time, making the system accessible to operational security teams.

The hardware component anchors these software guards to a tamper-resistant STM32C071RB microcontroller. HMAC-SHA256 token verification, replay attack detection via nonce tracking, and timestamp-based TTL enforcement all pass 100% of hardware validation trials, confirming that trust-transition checkpoints can be enforced at the silicon level independently of host software.

6.2 Future Scope

- **Live AWS integration:** Replace the demo mode with real `boto3` calls for live IAM graph construction and differential updates.
- **Multi-account federated graphs:** Extend loaders and graph representation to cross-account role trust chains spanning multiple AWS accounts.
- **GCP and Azure support:** Add loaders for GCP IAM and Azure RBAC to generalise to multi-cloud environments.

- **Hardware TPM integration:** Replace the STM32 HMAC engine with a TPM 2.0 module for OS-attested, standardised trust-transition enforcement.
- **RL-based weight adaptation:** Use a reinforcement learning agent to optimise ensemble weights directly against containment success rate on live breach simulations.

6.3 Learning Outcomes

- Designed and implemented a full six-module research pipeline in Python using object-oriented architecture, abstract base classes, dataclass schemas, and 381-test pytest coverage.
- Applied directed graph theory, Laplacian eigendecomposition, BFS, SIR epidemic modelling, and Monte Carlo percolation to a real-world cloud security problem.
- Implemented and benchmarked six fundamentally different propagation algorithms, and built an adaptive ensemble that consistently outperforms the best individual model.
- Built a full-stack web application: Flask REST API with SSE streaming, Three.js 3D interactive dashboard, multi-tab UI with admin panel and real-time breach simulation.
- Programmed an STM32 ARM Cortex-M0+ microcontroller in C to implement HMAC-SHA256 token verification and replay-attack protection, integrated over serial with a Python host.
- Parsed and validated real-world AWS IAM, Kubernetes RBAC, and Terraform HCL2 configurations, and validated the system against the CloudGoat penetration-testing benchmark suite.



Appendix A

Code Listings

APPENDIX A

CODE LISTINGS

A.1 Python Host: Token Construction and Hardware Validation

```
import hmac, hashlib, struct, serial, time

SECRET_KEY = b"my_secret_key_32bytes_padded!!!!"

def build_token():
    message = struct.pack(
        "<16s8s8sBBIH8sBB",
        b'\x01'*16,    # token_id
        b'\x02'*8,     # origin_hash
        b'\x03'*8,     # target_hash
        4, 6,          # depth, max_depth
        int(time.time()), 300, # timestamp, TTL
        b'\x04'*8, 1, 1    # nonce, edge_type, strictness
    )
    mac = hmac.new(SECRET_KEY, message, hashlib.sha256).digest()
    return message + mac    # 50 + 32 = 82 bytes

ser = serial.Serial('COM6', 115200, timeout=2)
time.sleep(2)
ser.write(b'T' + struct.pack("<I", int(time.time())) # time-
sync

payload = build_token()
for b in payload:
    ser.write(bytes([b]))
    time.sleep(0.001)
time.sleep(1)
print("STM:", ser.read_all().decode())    # ACCESS OK
```

```
for b in payload:
    ser.write(bytes([b]))
    time.sleep(0.001)
time.sleep(1)
print("STM:", ser.read_all().decode())    # REPLAY DETECTED
```

A.2 Running the Full TrustField Pipeline

```
from trustfield.ensemble.orchestrator import
    TrustFieldOrchestrator
from trustfield.loaders.aws_iam_loader import IAMPolicyLoader

# Load a real IAM JSON file
graph = IAMPolicyLoader().load("account_dump.json")

# Run the full 6-module pipeline
result = TrustFieldOrchestrator().analyze(
    graph, seed_nodes=["user-dev"]
)

print("Topology:           ", result.topology_type)
print("Ensemble scores:    ", result.node_scores)
print("PBR nodes:           ", result.blast_radius.pbr_nodes)
print("VBR nodes:           ", result.blast_radius.vbr_nodes)
print("Gap class:           ", result.blast_radius.
    gap_classification)
print("Guards deployed:     ", len(result.containment.
    blocked_transitions))
```

BIBLIOGRAPHY

- [1] M. Koch, L. V. Mancini, and F. Parisi-Presicce, “Graph-based specification of access control policies,” *Journal of Computer and System Sciences*, vol. 71, no. 1, pp. 1–38, 2005. DOI: 10.1016/j.jcss.2004.11.007
- [2] H. Yin, C. Gao, X. Yan, W. Chen, W. Li, and J. Tang, “Heterogeneous graph semi-supervised learning for access control decision,” *World Wide Web*, vol. 27, no. 2, pp. 1–23, 2024. DOI: 10.1007/s11280-024-01247-y
- [3] L. Zhou, H. Zheng, A. Hu, and G. Chen, “Decentralized federated graph learning with lightweight zero trust architecture for next-generation security,” *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 1, pp. 1–16, 2025. DOI: 10.1109/JSAC.2024.3481870
- [4] H. Mohamed, S. Mehrotra, and N. Venkatasubramanian, “An extended authorization policy for graph-structured data with complex relationship types,” *SN Computer Science*, vol. 2, no. 3, pp. 1–18, 2021. DOI: 10.1007/s42979-021-00618-8
- [5] S. Z. R. Rizvi and P. W. L. Fong, “Efficient authorization of graph-database queries in an attribute-supporting rebac model,” *ACM Transactions on Privacy and Security*, vol. 23, no. 4, pp. 1–33, 2020. DOI: 10.1145/3386898
- [6] N. Mahabadi, H. Alinejad-Rokny, M. Dehghani, et al., “Network properties for robust multilayer infrastructure systems: A review of percolation theory,” *IEEE Access*, vol. 9, pp. 95 397–95 411, 2021. DOI: 10.1109/ACCESS.2021.3092890
- [7] C. H. Nwokoye and N. Madhusudanan, “Epidemic models for malicious-code propagation in wireless sensor networks: An indispensable review,” *Wireless Personal Communications*, vol. 125, no. 3, pp. 2131–2167, 2022. DOI: 10.1007/s11277-022-09612-6
- [8] I. D. Mienye and Y. Sun, “A survey of ensemble learning: Concepts, algorithms, applications, and prospects,” *IEEE Access*, vol. 10, pp. 99 129–99 149, 2022. DOI: 10.1109/ACCESS.2022.3207287
- [9] Z. Wang, C. Liu, C. Wang, and Z. Guan, “From proof-of-concept to exploitable: A study of real-world exploitability analysis,” *Cybersecurity*, vol. 2, no. 1, pp. 1–15, 2019. DOI: 10.1186/s42400-019-0042-x

- [10] T. Shakeel, S. Habib, A. Wasim, M. A. Alowidbi, W. A. Khan, and M. Ghaly, “A survey of data visualization tools: Challenges, limitations, and future directions,” *IEEE Access*, vol. 10, pp. 107 526–107 562, 2022. DOI: 10 . 1109 / ACCESS . 2022 . 3212500

